



Hiram Finance

BOND INDEX TRACKING WITH LOCAL SEARCH TECHNIQUES

Author : Paul Demey

Date : 24/01/2024

Contents

Introduction	2
1 Literature review	3
2 Problem formalization	5
2.1 Bond universe and notations	5
2.2 Objective function	6
3 Algorithms description	7
3.1 Neighborhood definition	7
3.2 Hill Climbing	8
3.3 Tabu search	8
3.4 Simulated annealing	9
4 Algorithms comparison	9
4.1 Study of convergence	10
4.2 Algorithms performance	13
4.3 Comparison of portfolios	14
5 Tabu extended in high dimension	15
6 Portfolio rebalancing	17
Conclusion	19
References	20



Introduction

Within asset management companies, index management is a department managing funds passively: its aim is to replicate the performance of a benchmark index, so that the fund's return is as close as possible to that of the benchmark. These indices can be equity (such as Eurostoxx 50, CSI 300 or Topix), sovereign bond (Iboxx Euro Sovereign) or credit (Iboxx Eur Corporate).

In theory, to replicate a benchmark, the fund could hold all its component in identical proportions, in which case replication would be perfect. In practice, however, full replication is often difficult to achieve. If the number of securities in the benchmark is large, total replication generates high transaction costs (whereas index management seeks to keep costs as low as possible); when the fund experiences small subscriptions or redemptions, the manager would have to deal with each security for a low nominal amount, potentially below the minimum tradable on the market (if any). As a result, index managers frequently resort to sampling, which consists in holding only a limited number of securities. The challenge is to determine which securities to hold, and in what proportion of the total outstanding, so as to stay as close as possible to the benchmark.

To measure the distance between a portfolio and its benchmark index, the approach differs depending on the asset class under consideration. On equity markets, it is traditionally measured by the ex-ante Tracking Error Volatility (TEV), which represents the expected volatility of the return's difference, calculated from the variance-covariance matrix of the securities and the weights of each security. In the case of a bond index (sovereign or credit securities), this approach is no longer possible, as bond volatility is not stable over time, and the assumption of stationarity of covariances is no longer appropriate. Instead, we define the distance between two portfolios as the difference between their characteristics : duration, sector allocation, duration time spread, etc. We'll look at this issue in the section 2.2.

Once the distance between two portfolios has been defined, the best replicating portfolio is found by minimizing the distance under a set of constraints. From a computational point of view, finding the portfolio that minimizes this distance, without imposing a constraint on the number of securities can be done efficiently using convex optimization algorithms. However, the introduction of a cardinality constraint represents a computational challenge, due to the exponential number of combinations, and the presence of numerous local minima. It's a problem known to be NP-hard, which cannot be solved - today - in polynomial time. There are



many possible approaches to this problem, and the literature on the subject is relatively extensive. We'll take a look at some of these approaches in the section 1.

Most studies focus on the equity asset class. In this document, on the contrary, we will look at the case of replicating a bond index. Bond market has specificities that need to be taken into account: first, as mentioned above, the measurement of the distance between two portfolios differs, and second, there is a minimum tradable and a lot size. These particularities, described in [Ben Slimane (2021)], will lead us to consider a discrete set for the detained bond quantities.

We will focus on local exploration methods, and define different types of neighborhoods. We will compare several local exploration algorithms (Hill Climbing, Tabu Search, Simulated Annealing), and compare their efficiency, in terms of computation time and quality of the solution obtained.

1 Literature review

Let's consider the tracking error minimization problem under minimum and maximum weight constraints. The addition of a cardinality constraint transforms it into an NP-hard problem, as shown by [Mutunge et al. (2018)]. Indeed, they show that there is a polynomial reduction of the independent vertex set problem, which is NP-complete. No solution algorithm is therefore known to run in polynomial time. As soon as the dimension of the problem is significant, it is illusory to look for an exact solution. The techniques used are therefore approximate solutions. We will distinguish methods solving the problem with an approximation, and meta-heuristics aiming at approaching the exact solution.

In the first category, [Roncalli (2021)] mentions two algorithms: "backward elimination", which proceeds iteratively, starting with a portfolio composed of all stocks, and eliminating at each stage the stock with the highest tracking error; and "forward selection", which starts with an empty portfolio, adding to it at each stage the stock with the lowest TEV. Both these algorithms provide a solution, albeit potentially far from the optimum. In a similar way, [Sakurai et al. (2023)] seek to solve cardinality-constrained optimization in two stages, with a particular focus on the stock selection stage. The selection uses the market capitalization of the securities and the correlation between all the securities.

[Schaerf (2002)] used local search techniques applied to stock indices. The technique appears promising, but is applied to stock indices, with a relatively small



size (up to 200 stocks).

An alternative proposed by [Coleman et al. (2004)] is to approximate the (discontinuous) stock count function by a continuously differentiable function. The method is iterative: initially, the function is convex but relatively far from the real function, then becomes non-convex (but closer to the real function) over the course of iterations. At each stage, a linearly constrained quadratic optimization problem must be solved, which classical algorithms handle efficiently.

[Mutunge et al. (2014)] use relaxation methods (Lagrangian relaxation, continuous relaxation) to obtain a lower bound on the optimal TE. In a second step, and to obtain an upper bound (approximate solution), they use heuristics derived from a greedy algorithm: at each iteration, they add the security that minimizes the TE (computed by a quadratic programming algorithm), or exchange two tracks if the resulting TE is improved. Their method requires a quadratic program to be solved at each iteration, and demands relatively long computation times.

[Tadonki et al. (2014)] propose to solve the problem using a Branch and Bounds algorithm combined with a Blenders decomposition. The Blenders decomposition separates the problem into two sub-problems. The Branch and Bounds algorithm is useful for solving combinatorial optimization problems. The principle is to construct a search tree which is explored by removing unnecessary / dominated branches.

[Mutunge et al. (2018)] present a two-stage heuristic: build an optimal portfolio by adding securities one by one so as to minimize the TE, and then trade securities as long as the objective function is improved. Each iteration requires continuous optimization, which results in relatively long computation times.

[Ben Slimane et al. (2023)] seek to replicate a bond index. To define the distance between two portfolios, they use the L1 norm of the vector of feature spreads (duration, duration time spread, etc.). This brings them back to a MILP (Mixed Integer Linear Programming) problem, which algorithms like Branch and Cut (a combination of Branch and Bound and Cutting Planes) solve efficiently.

A second class of papers uses population-based meta-heuristics: genetic algorithms, or swarm optimization. [Beasley et al. (2003)] experiment genetic algorithms whereas [Yuen et al. (2021)] study both swarm optimization or genetic algorithm. The article is limited to relatively low dimensions. Finally, [Ben Slimane (2021)] uses genetic algorithm as well, but with a focus on bond index replication. The problem is to find the optimal rebalancing for a bond portfolio that replicates a benchmark. Purchases/sales are made in batches, which leads to combinatorial optimization

(discrete variables).

Thus, two papers focus on the specific case of bond replication: [Ben Slimane (2021)] with a genetic algorithm and [Ben Slimane et al. (2023)] using a Branch and Cut method.

Since local search algorithms applied to bond tracking have not been explored, we wish to study them here, in order to find out whether they provide acceptable solutions in reasonable time. In particular, we'll be looking at the definition of the neighborhood of a portfolio. Several algorithms will be considered and compared, both in terms of computation time and quality of the solution obtained.

2 Problem formalization

2.1 Bond universe and notations

Consider a universe of N bonds characterized by :

- a price vector P with dimensions $(N, 1)$
- a duration vector D with dimensions $(N, 1)$
- a partial durations matrix DP with dimensions (N, K) where K is the number of duration buckets
- a sector allocation matrix RS with dimensions (N, S) where S is the total number of sectors, such that $RS_{i,s} = 1$ if bond i belong to sector s and 0 otherwise
- a sector duration matrix DS with dimensions (N, S) such that $DS_{i,s} = D_i$ if bond i belong to sector s and 0 otherwise

Partial durations $(DP)_{ik}$ represent a decomposition of duration D_i over K maturity pillars $\{1Y, 2Y, 5Y, 7Y, 10Y, 20Y\}$ so that $\sum_k (DP)_{ik} = D_i$, and give the sensitivity of the price of bond i to shocks on different parts of the yield curve.

Other characteristics could be added, depending on the objective of the asset manager, such as duration-time-spread, yield-to-maturity, a liquidity indicator, etc. However, for sake of clarity, we chose to limit the number of characteristics.

We wish to track a portfolio made up of N securities, in proportion W^* . This portfolio has the following characteristics:



- duration : $D^\top W^*$
- partial durations vector : $DP^\top W^*$
- sector allocation vector : $RS^\top W^*$
- weighted sector duration vector: $DS^\top W^*$
- proportion of cash : $cash(W^*) = 1 - sum(W^*)$

2.2 Objective function

We measure the distance between two bond portfolios as the sum of the deviations on each of the explanatory factors. Thus, the distance between two portfolios W_1 and W_2 is the weighted-sum of the characteristics difference :

$$\begin{aligned} Delta(W_1, W_2) = & \alpha_1 \left| D^\top (W_1 - W_2) \right| + \alpha_2 \left\| DP^\top (W_1 - W_2) \right\| + \alpha_3 \left\| RS^\top (W_1 - W_2) \right\| \\ & + \alpha_4 \left\| DS^\top (W_1 - W_2) \right\| + \alpha_5 |cash(W_1) - cash(W_2)| \end{aligned}$$

where $\|x\|$ is the norm of vector x .

To the objective of minimizing this deviation, we add a penalty to make sure that the number of securities with a non-zero weight $N(W)$ is less than a limit N_{max} . Thus, the objective function of the minimization program is :

$$Cost(W) = Delta(W, W^*) + \alpha_6 (N(W) - N_{max})^+$$

The weights α_i must be adjusted according to the preferences one wish to assign to each objective.

In the bond market, bonds can be traded with a minimum tradable quantity, and above this amount, by lot sizes. For example, in the EUR credit market, the minimum tradable quantity is generally EUR 100,000 and the lot size EUR 1,000. However, in practice, the quantities traded are mostly multiples of the tradable minimum (see [Ben Slimane (2021)]). This is what we'll assume from now: the quantities processed are a multiple of the minimum tradable, so our optimization is based on discrete variables. If X is the vector of lot quantities held and NAV is

the NAV of the fund :

$$\begin{aligned} W_i &= \frac{P_i X_i \delta}{NAV} \\ X_i &\in \mathbb{N} \\ X_i &\leq X_{max} \end{aligned}$$

Hence the optimization program is:

$$\begin{aligned} \min_X \quad & \text{Delta}(W, W^*) + \alpha_6 (N(W) - N_{max})^+ \\ \text{s.t.} \quad & W_i = \frac{P_i X_i \delta}{NAV} \\ & X_i \in \mathbb{N} \\ & X_i \leq X_{max} \end{aligned}$$

3 Algorithms description

We study three local search algorithms: Hill Climbing, Tabu search and Simulated Annealing. The principle of local search is to start with an initial portfolio (a vector X_0 containing the number of lots of each security), search for a portfolio in the neighborhood that improves the objective function, and repeat the operation until a minimum (local or global) is reached. Before presenting these algorithms in detail, we first define the neighborhoods that will be used.

3.1 Neighborhood definition

For local search algorithms to work, the neighborhood must be "connected", i.e. it must be possible, starting from a portfolio X , and through a succession of neighbors, to reach any other portfolio in the universe.

Simple neighborhood

A possible connected neighborhood, which we'll call V_{simple} , is made up of the $2N$ neighbors obtained by adding or subtracting one lot on each bond.

$$V_{simple} \left(\begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{pmatrix} \right) = \underbrace{\left(\begin{pmatrix} X_1 + 1 \\ X_2 \\ \vdots \\ X_N \end{pmatrix}, \begin{pmatrix} X_1 \\ X_2 + 1 \\ \vdots \\ X_N \end{pmatrix}, \dots, \begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_N + 1 \end{pmatrix} \right)}_N, \underbrace{\left(\begin{pmatrix} X_1 - 1 \\ X_2 \\ \vdots \\ X_N \end{pmatrix}, \begin{pmatrix} X_1 \\ X_2 - 1 \\ \vdots \\ X_N \end{pmatrix}, \dots, \begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_N - 1 \end{pmatrix} \right)}_N$$



This neighborhood is minimalist (i.e. the smallest connected neighborhood), so the time required to explore the neighborhood is low. But the smaller the neighborhood, the more difficult it is to escape from a local minimum. We therefore also consider a larger neighborhood in the following paragraph.

Extended neighborhood

We introduce all bond exchanges by defining V_{swap} as all the portfolios obtained by selling k lots on one bond and simultaneously buying k lots on another. The number of portfolios is $2X_{max}C_N^2$ (at a fixed k , there are $2C_N^2$ ways of exchanging two securities). As soon as N is large (say $N > 1000$), the neighborhood V_{swap} cannot be calculated in reasonable time. We therefore define a random neighborhood $V_{swap}(v)$ made up of v neighbors, randomly chosen from V_{swap} . The extended neighborhood $V_{extended}$ is obtained as the union of V_{simple} and $V_{swap}(p)$.

$$V_{extended} = V_{simple} \cup V_{swap}(v)$$

This neighborhood is connected, since he contains V_{simple} .

3.2 Hill Climbing

The Hill Climbing method consists in starting from an initial portfolio, evaluating neighbors portfolios, choosing the best of these, and repeating until the stopping criterion (number of iterations, computation time, etc.) is reached. This is the equivalent, in a discrete universe, of gradient descent optimization in a continuous universe. The algorithm converges on a local optimum.

To reduce the number of local optima, one solution is to enlarge the neighborhood. But in return for a larger neighborhood, computation time also increases. Hence Hill Climbing can use the simple neighborhood (referred to as HC simple) or extended neighborhood (referred to as HC extended).

3.3 Tabu search

Hill Climbing does not allow to escape from a local minimum. Indeed, when the local minimum is reached, all neighbors have a higher value, and the algorithm stays on the local minimum. The Tabu method offers a way out of local minima by prohibiting previously-visited positions over the last iterations. These are stored in a list known as a "tabu list". The longer the list, the easier it is to escape from a local minimum, but the longer the computation time. There is no method for



calibrating the length of the tabu list; it is generally set empirically.

3.4 Simulated annealing

Simulated annealing is inspired by the annealing process used in metallurgy. The aim is to escape from local minima. In our case, we will use a simple neighborhood (an extended neighborhood is not necessary since SA implicitly visits a large neighborhood already). It starts with an initial portfolio, and randomly selects a neighbor from the neighborhood. If the objective function is improved, this neighbor is retained. Otherwise, it is retained only with a probability $p = e^{-\frac{\Delta E}{T}}$ which depends on two parameters T and ΔE . By analogy with the physical process, the fictitious parameter T corresponds to the system temperature. It evolves deterministically, decreasing at each iteration. The most commonly used law for temperature decay is $T_{i+1} = \lambda T_i$ where $0 < \lambda < 1$. The change in system energy E corresponds to the change in the function to be minimized. Thus, a solution that worsens the objective function will be all the more easily accepted if (i) the objective function is only slightly degraded and (ii) the temperature is high.

4 Algorithms comparison

The principle of our simulations is to test algorithms on a randomly constructed benchmark portfolio, rather than on real data. The advantage is twofold: first, we can construct a multitude of random benchmarks and test our algorithms on each of them, to see how they perform in a variety of configurations; second, we can easily run tests with benchmarks of various sizes.

We create a universe of bonds, and then build a "solution" portfolio, with a number of stocks $N^* \leq N_{max}$. This portfolio is used to parametrize the objective function: we deduct from it the target duration, target sector allocation, target partial duration, etc. This procedure guarantees the existence of a "solution" portfolio that cancels the objective function. If an algorithm results in a portfolio that does not cancel the objective function, the discrepancy is due to the algorithm itself, and not to the fact that it is not possible to find a solution.

In detail, we proceed as follows:

1. Random construction of a universe of N bonds: price, maturity, duration, partial durations, sector.



2. Calculation of characteristic matrices D, DP, RS, DS
3. Random construction of a "solution" portfolio made up of cash, $N^* \leq N_{max}$ bonds, with weights W^*
4. From the portfolio W^* , determine target quantities :
 - (a) target duration $D^\top W^*$
 - (b) target partial duration $DP^\top W^*$
 - (c) target sector allocation $RS^\top W^*$
 - (d) target sectorial duration $DS^\top W^*$
 - (e) Target cash $Cash(W^*)$
5. Find the portfolio W that minimizes the $Cost(W)$ function

If the portfolio W obtained in step 5 equals W^* , the optimization is perfect. Otherwise, the value of $Cost(W)$ quantifies the deviation from the "solution" portfolio.

4.1 Study of convergence

The Figure 1 shows the convergence of each of the algorithms: Hill Climbing with simple neighborhood, Hill Climbing with extended neighborhood, Tabu with simple neighborhood, Tabu with extended neighborhood.

The parameters used are : $N_{univers} = 100$; $N_{max} = 20$; $X_{max} = 10$; $\alpha = (1, 0.5, 0.5, 1, 1, 10)$; $Cash(W^*) = 1\%$; $v = 500$ and the initial portfolio is zero.

The value of the objective function is plotted against the number of iterations. The graph on the right ("Zoom out") shows all iterations, while the graph on the left starts at 100th iteration.

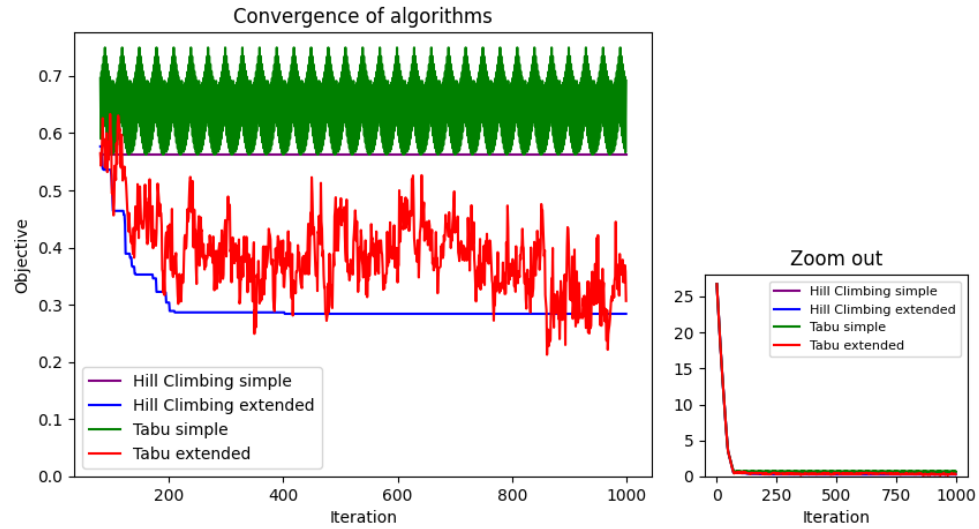


Figure 1: Convergence of algorithms

We note that :

- the simple Hill Climbing algorithm converges quickly to a local minimum;
- the simple Tabu algorithm escapes from the same local minimum but returns to it regularly (oscillations);
- Extended Neighborhood allows Hill Climbing and Tabu to reach and exit local minima.
- Tabu extended gives the best result.

To obtain a better solution with an algorithm like simple Hill Climbing, we repeat the algorithm for several starting portfolios. For each of them, a different local minimum is obtained. However, in our case, as illustrated by the Figure 2, local minima of the optimized function are too numerous, and the solutions are all far from the optimal portfolio (objective function > 1).

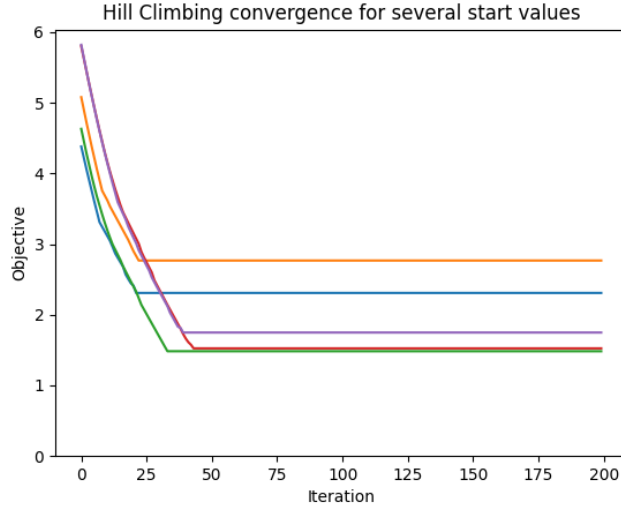


Figure 2: Hill Climbing convergence for several starting portfolios

The Figure 3 shows the convergence by simulated annealing. During the first iterations (up to 1500), the objective function is increasing, because the initial temperature is high, which leads to selecting worse portfolios with a high probability. When the temperature decreases (iterations > 1500), the objective function decreases with temperature.

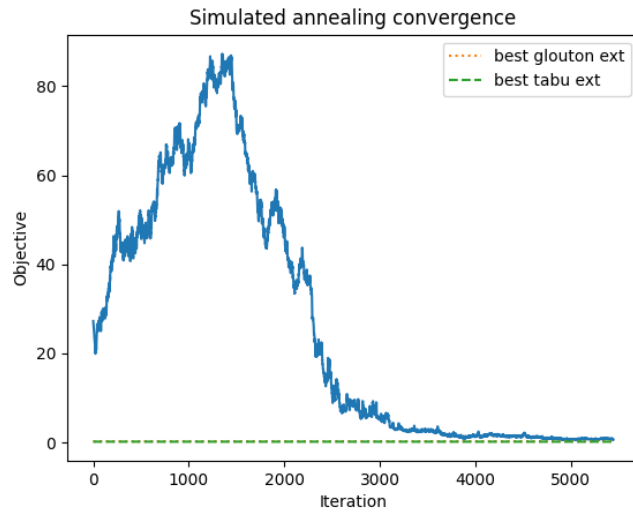


Figure 3: Simulated annealing convergence

4.2 Algorithms performance

The portfolio search operation is repeated for fifty randomly selected target portfolios, to obtain the distribution of the objective function for each algorithm, which is shown in Figure 4. The table below also gives the values of the means and standard deviations.

	HC	HC ext.	Tabu	Tabu ext.	S. Annealing
Mean	1.28	0.59	1.22	0.16	0.42
Stdev	0.24	0.18	0.24	0.04	0.14

For example, for simple Hill Climbing, the mean value of the objective function is 1.28, and the standard deviation is 0.24. We'll see below how 'far' is a distance of 1.28, in terms of characteristics. But we conclude that:

- 'extended' neighborhoods are much more efficient than "simple" ones. In Hill climbing, an extended rather than a simple neighborhood improves the objective from 1.28 to 0.59; with Tabu Search, it drops from 1.22 to 0.16.
- Tabu Search is more efficient than Hill Climbing, especially with an extended neighborhood.
- Simulated annealing, while better than simple neighborhood algorithms, doesn't perform as well as extended Tabu.
- Extended Tabu performs best

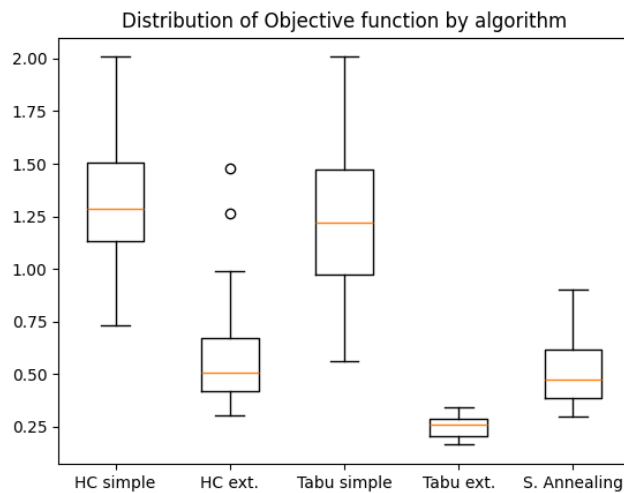


Figure 4: Distribution of objective functions

4.3 Comparison of portfolios

The average objective functions calculated above vary between 0 and 2, depending on the algorithm. To find out how far are these portfolios from the target one, we list the characteristics obtained for one optimization below. The target portfolio contains 15 bonds, 1% cash, and has a duration of 11.56.

The "Tabu extended" portfolio, for example, was calculated in 7.8 seconds, contains 19 bonds (for a maximum limit of 20) and 0.92% cash (for a target of 1%), and an objective function value of 0.21.

Portfolio	Exec time	Objective function	Nb bonds	Duration	Cash in %
Target	0.00	0.00	15.00	11.56	1.00%
Hill Climbing simple	4.36	0.56	20.00	11.57	1.05%
Hill Climbing extended	6.81	0.28	20.00	11.56	0.96%
Tabu simple	5.43	0.56	20.00	11.57	1.05%
Tabu extended	7.87	0.21	19.00	11.55	0.92%
S.Annealing	8.37	0.64	20.00	11.61	1.13%

The table below summarizes the partial durations: for example, the 2-year target partial duration is 0.67, while for Tabu extended it is 0.68.

Partial duration	Target	Hill Climbing simple	Hill Climbing extended	Tabu simple	Tabu extended	S.Annealing
2Y	0.67	0.78	0.72	0.78	0.69	0.68
5Y	0.61	0.66	0.72	0.66	0.71	0.60
7Y	0.84	0.62	0.72	0.62	0.84	0.94
10Y	0.47	0.47	0.47	0.47	0.46	0.47
15Y	3.21	3.11	3.20	3.11	3.11	3.22
20Y	5.76	5.93	5.72	5.93	5.75	5.70

The sector breakdown of the solution obtained by each algorithm is shown below. Whatever the algorithm, the sector allocation is relatively close to the target portfolio.

Sector weight (%)	Target	Hill Climbing simple	Hill Climbing extended	Tabu simple	Tabu extended	S.Annealing
Agencies	9%	9%	10%	9%	10%	8%
Auto	0%	3%	0%	3%	0%	2%
Banking	0%	2%	0%	2%	0%	0%
Capital Goods	0%	0%	0%	0%	0%	0%
Chemistry	13%	8%	11%	8%	8%	8%
Consum	16%	16%	14%	16%	12%	23%
Energy	5%	3%	5%	3%	6%	3%
Insurance	0%	0%	0%	0%	0%	0%
Insurance Sub	0%	0%	0%	0%	0%	0%
Lower Tier2	10%	11%	10%	11%	9%	13%
Others	11%	11%	11%	11%	11%	13%
Pfandbriefe	11%	12%	15%	12%	15%	13%
Spec finance	10%	12%	14%	12%	14%	16%
Telecom/Techno	3%	1%	3%	1%	3%	0%
Transportation	0%	0%	0%	0%	0%	1%
Utility	10%	9%	6%	9%	10%	0%

Finally, the following table shows sector-weighted duration. The proximity of the extended Tabu portfolio to the target portfolio is particularly striking.

Sect. Dur. * weight	Target	Hill Climbing simple	Hill Climbing extended	Tabu simple	Tabu extended	S.Annealing
Agencies	0.61	0.48	0.62	0.48	0.63	0.83
Auto	0.00	0.46	0.00	0.46	0.00	0.23
Banking	0.00	0.20	0.00	0.20	0.00	0.00
Capital Goods	0.00	0.00	0.00	0.00	0.00	0.00
Chemistry	1.27	1.43	1.36	1.43	1.22	1.43
Consum	2.00	2.00	1.96	2.00	2.05	2.19
Energy	0.42	0.32	0.43	0.32	0.40	0.20
Insurance	0.00	0.00	0.00	0.00	0.00	0.00
Insurance Sub	0.00	0.00	0.00	0.00	0.00	0.00
Lower Tier2	1.60	1.47	1.46	1.47	1.60	1.53
Others	1.57	1.57	1.57	1.57	1.57	1.18
Pfandbriefe	1.87	1.72	1.81	1.72	1.89	1.93
Spec finance	1.79	1.62	1.90	1.62	1.77	1.93
Telecom/Techno	0.29	0.18	0.37	0.18	0.29	0.00
Transportation	0.00	0.00	0.00	0.00	0.00	0.15
Utility	0.13	0.12	0.08	0.12	0.13	0.00

5 Tabu extended in high dimension

Previous results show that Tabu Extended is the best performing among tested algorithms. We'd like to illustrate its efficiency in high dimension, i.e. when the number of bonds in the universe is high. We simulate a 3000 bonds universe, and seek to replicate a target index, with a maximum of 100 stocks. The parameters used are : $N_{universe} = 3000$; $N_{max} = 100$; $X_{max} = 10$; $\alpha = (1, 0.5, 5, 1, 1, 10)$; $Cash(W^*) = 1\%$; $v = 100$.

The value of the objective function is shown as a function of execution time in Figure 5. Within a reasonable time (15 to 20 minutes), efficient sampling is achieved.



Improvement is very poor afterwards. Note that it would be useful to define an optimization stopping criterion. For example, when the objective function reaches a predefined threshold, or when the objective function has not been improved for a certain number of iterations.

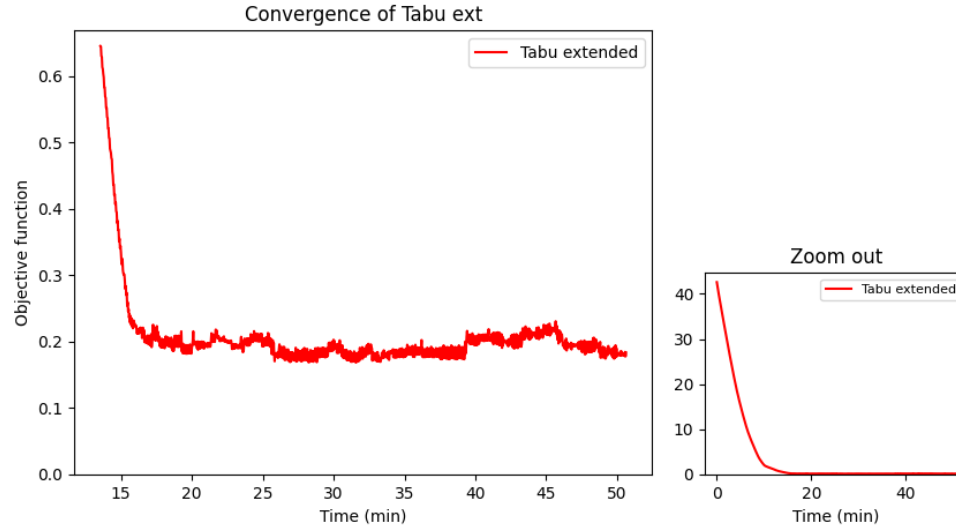


Figure 5: Tabu extended in high dimension

To give an idea of the closeness of the resulting portfolio, the tables below show the characteristics of the sampled portfolio and those of the target portfolio.

The constraint in number of securities is almost saturated (99 securities selected for a constraint of 100), and has a duration of 9.16, as expected.

Portfolio	Exec time (min)	Objective function	# bonds	Duration	Cash in %
Target	0.00	0.00	95.00	9.16	1.00%
Tabu extended	50.66	0.17	99.00	9.16	1.00%

The partial durations are almost identical to the target partial durations:

Partial dur.	Target	Tabu extended
2Y	0.65	0.65
5Y	0.79	0.79
7Y	0.74	0.74
10Y	1.23	1.23
15Y	3.14	3.14
20Y	2.61	2.61

and sector breakdown, as well as sector durations, are close to those expected.

Sector dur. * weight	Target	Tabu extended	Sector weight (%)	Target	Tabu extended
Agencies	0.53	0.48	Agencies	6%	5%
Auto	0.52	0.52	Auto	5%	5%
Banking	0.81	0.79	Banking	7%	8%
Capital Goods	0.53	0.55	Capital Goods	10%	10%
Chemistry	1.20	1.16	Chemistry	14%	13%
Consum	0.53	0.50	Consum	4%	5%
Energy	0.65	0.63	Energy	6%	6%
Insurance	0.85	0.90	Insurance	9%	9%
Insurance Sub	0.28	0.40	Insurance Sub	4%	4%
Lower Tier2	0.33	0.37	Lower Tier2	5%	5%
Others	0.45	0.44	Others	4%	5%
Pfandbriefe	0.50	0.60	Pfandbriefe	6%	5%
Spec finance	0.42	0.40	Spec finance	3%	3%
Telecom/Techno	0.47	0.41	Telecom/Techno	4%	5%
Transportation	0.30	0.23	Transportation	4%	4%
Utility	0.80	0.78	Utility	8%	7%

In high dimensions (3000 stocks), the algorithm delivers portfolios close to the target, in a reasonable time (20 minutes).

6 Portfolio rebalancing

In previous sections, we studied the case of building a portfolio from cash. In practice, it is also often necessary to rebalance a portfolio, i.e. to buy and sell securities following subscriptions and redemptions in an existing fund. Typically, if a fund receives a subscription of 4% of its NAV, the manager will seek to determine the operations to be carried out to replicate the index, under cardinality constraints, and under turnover constraints (to limit sales of securities in the case of a subscription).

Determining the optimal rebalancing is a simpler problem than replicating from cash. Indeed, when optimizing in a discrete universe, the number of possible combinations is reduced, since the amount to be distributed over the securities in the universe is smaller.

To illustrate this, we show in Figure 6 the distribution of the objective functions, in the case of a subscription of 4% of the amount outstanding, and with the following parameters: $N_{universe} = 100$; $N_{max} = 20$; $X_{max} = 20$; $\alpha = (1, 0.5, 0.5, 1, 1, 10)$; $Cash(W^*) = 1\%$; $v = 500$. Only bonds purchases are allowed. The ranking of the algorithms is identical to the general case (ie Tabu extended is the best performer), but the value of objective functions are lower.



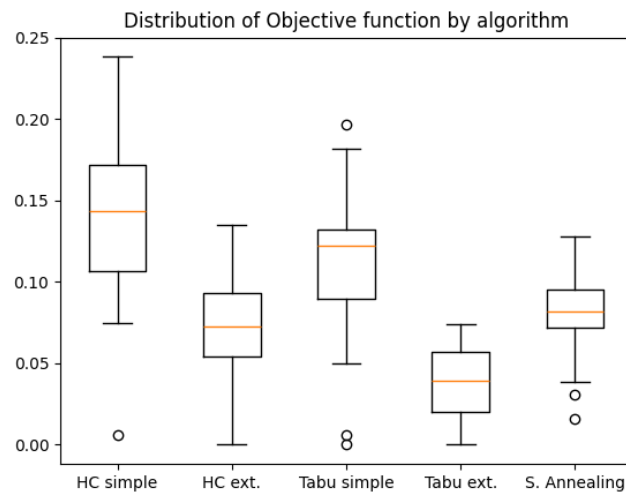


Figure 6: Distribution of objective functions - portfolio rebalancing

Conclusion

Sampling a bond index can be reduced to solving a mixed integer quadratic optimization problem. If we consider that the quantities traded on the bond market are always a multiple of a lot size, it can be seen as a quadratic integer programming problem, which is NP-complete. Various approaches are proposed in the literature. We focus here on local search methods, with an exploration of the neighborhood definitions. The local search approach has the advantage to handle any type of constraints, linear or not.

The classic Hill Climbing technique, with a *simple* neighborhood, is not efficient, because the function to be optimized contains many local optima in which it gets stuck. Simulated annealing is a way to escape these local minima, but the speed convergence is slow.

We proposed an alternative which is the combination of a Tabu search with an *extended* neighborhood. It appears to be effective, compared to others tested algorithms. Even in high dimension (sampling of a 3000-bonds index), we reach a good quality approximate solution, in a reasonable time.

To further improve the convergence speed, other types of neighborhoods could be considered. Variable neighborhood search are recommended in [Devarenne (2007)] and an application to bond index sampling might be useful. The idea is to modify the definition of the neighborhood during the optimization process, depending on the current state. The basic variable neighborhood, for instance, detects when a local optimum is reached. The neighborhood is extended until the local minimum is escaped, and it is reduced when this is achieved.

Finally, a further interesting study would be to benchmark our algorithm against methods proposed in the literature, such as Genetic Algorithm or Branch and Cut, in terms of quality of the solution, and execution time.



References

- [Beasley et al. (2003)] Beasley J.E., Meade N., Chang T.J., «Evolutionary heuristic for the index tracking problem», 2003
- [Ben Slimane (2021)] Ben Slimane M., «Bond Index Tracking with Genetic Algorithms», Amundi Working Paper, 2021
- [Ben Slimane et al. (2023)] Ben Slimane M., Menchaoui G., «Bond Portfolio Optimization and Mixed Integer Programming», Amundi Working Paper, 2023
- [Coleman et al. (2004)] Coleman T., Li Y., «Minimizing Tracking Error While Restricting the Number of Assets», 2004
- [Devarenne (2007)] Devarenne I., «Etude en recherche locale adaptative pour l'optimisation combinatoire», Thèse de doctorat en Informatique, 2007
- [Mutunge et al. (2014)] Mutunge P., Haugland D., «The index tracking problem with a limit on portfolio size», University of Bergen, 2014
- [Mutunge et al. (2018)] Mutunge P., Haugland D., «Minimizing the tracking error of cardinality constrained portfolios», Computers and Operations Research, 2018
- [Roncalli (2021)] Roncalli T., «Asset Management, Lecture 1. Portfolio Optimization», University of Paris Saclay, 2021
- [Sakurai et al. (2023)] Sakurai Y., Wakabayashi D., Ishizaki F., «Formulations to select assets for constructing sparse index tracking portfolios», 2023
- [Schaerf (2002)] Schaerf A., «Local Search Techniques for Constrained Portfolio Selection Problems», Computational Economics, 2002
- [Tadonki et al. (2014)] Tadonki C., Vial J.P., «Portfolio selection with cardinality and bound constraints», 2014
- [Yuen et al. (2021)] Yuen M.C, Ng S.C., Leung M.F., «Metaheuristics for Sparse Index-Tracking Problem: A Case Study on



FTSE 100», Journal of Physics: Conference Series,
2021.

